

SUMMER RESEARCH INTERNSHIP REPORT

Computer Log Anomaly Detection Using Machine Learning Techniques

A Comparative Study Using Machine Learning and Deep
Learning Models

Submitted by

Rajol Kumar

Roll No.: 25MAC2R19

Under the Guidance of

Dr. Sri Vallabha Deevi

Adjunct Faculty

Wadhwani School of Data Science & Artificial Intelligence (WSAI)

Indian Institute of Technology Madras

Summer Research Internship

2026

Certificate

This is to certify that the internship report entitled **Computer Log Anomaly Detection Using Machine Learning Techniques** submitted by **Rajol Kumar** (Roll No. 25MAC2R19) is a genuine record of the research work carried out during the Summer Research Internship under my guidance at the Wadhvani School of Data Science and Artificial Intelligence (WSAI), Indian Institute of Technology Madras.

To the best of my knowledge, this work has not been submitted elsewhere for the award of any degree, diploma or certificate.

Dr. Sri Vallabha Deevi

Adjunct Faculty

Dept. of Data Science & AI

Indian Institute of Technology Madras

Acknowledgement

I express my sincere gratitude to my mentor, Sri Vallabha Deevi, for providing me the opportunity to undertake this Summer Research Internship at the Wadhwani School of Data Science and Artificial Intelligence (WSAI), Indian Institute of Technology Madras. His guidance, insightful suggestions, continuous encouragement, and constructive feedback contributed greatly to the successful completion of this work. Every discussion during the internship helped sharpen both my technical understanding and my research methodology. I also thank the faculty members and internship coordinators of WSAI for providing an excellent learning environment and the resources needed to carry out this research.

Finally, I am deeply grateful to my family and friends for their constant encouragement, patience, and support throughout the internship period.

Rajol Kumar

Abstract

The growing scale of Internet-connected systems and the increasing sophistication of cyber attacks have made network security a central concern for modern organisations. Signature-based intrusion detection is effective only against previously catalogued attacks and is unable to identify new or evolving threats [1]. Anomaly detection based on machine learning has therefore emerged as a complementary approach: rather than matching known attack signatures, it learns the statistical properties of normal behaviour and flags deviations from that behaviour as suspicious [2].

This internship studies machine learning and deep learning approaches to computer log anomaly detection across datasets of increasing size and realism. The investigation begins with a small, labelled network traffic dataset [3], on which supervised classifiers are compared against unsupervised anomaly detection methods to establish baseline intuitions. It then moves to two medium-scale benchmark datasets: UNSW-NB15 [4, 5], used to evaluate supervised classification under feature selection and hyperparameter optimisation [6], and CSE-CIC-IDS2018 [7, 8], used to compare four unsupervised anomaly detection algorithms under the same experimental protocol. The comparative results from both medium-scale studies motivate the choice of the Deep Autoencoder [9] as the model most capable of generalising to complex, high-dimensional network traffic without requiring labelled attacks.

The final stage of the internship investigates the Autoencoder in detail on a large subset of CSE-CIC-IDS2018 [7], comparing two training regimes: training exclusively on benign traffic, and training on traffic deliberately contaminated with a small proportion of attacks. Both regimes are evaluated against independent test sets with anomaly rates of 0.2%, 1%, and 5%, producing a systematic picture of how training data composition and attack prevalence jointly affect detection performance.

Overall, the results show that both hyperparameter optimisation [6] and feature selection [10] improve anomaly detection substantially, that the Autoencoder consistently outperforms statistical and distance-based alternatives on realistic network traffic, and that reconstruction-based detection remains robust under limited training contamination

even though its separation margin narrows as contamination increases. These findings offer practical guidance for deploying Autoencoder-based intrusion detection in environments where perfectly clean, fully labelled training data cannot be guaranteed.

Contents

Certificate	1
Acknowledgement	2
Abstract	3
1 Introduction	1
1.1 Introduction	1
1.2 Computer Logs	2
1.3 Network Traffic Logs	2
1.4 Network Anomaly Detection	3
1.5 Motivation	3
1.6 Problem Statement	4
1.7 Objectives	4
1.8 Major Contributions	4
1.9 Datasets Used	5
1.10 Report Organization	6
2 Literature Review	7
2.1 Introduction	7
2.2 Intrusion Detection Systems	7
2.3 Machine Learning for Anomaly Detection	8
2.3.1 Supervised Learning	8
2.3.2 Unsupervised Learning	8
2.4 Deep Learning for Network Anomaly Detection	9
2.5 Autoencoder-Based Anomaly Detection	9
2.6 Publicly Available Intrusion Detection Datasets	9
2.7 Summary of Previous Research	10
2.8 Research Gap	10

2.9	Chapter Summary	11
3	Datasets and Anomaly Detection Methods	12
3.1	Introduction	12
3.2	Datasets Used	12
3.3	Small Dataset	13
3.4	Medium Datasets	13
3.4.1	UNSW-NB15	13
3.4.2	CSE-CIC-IDS2018 (Medium Subset)	14
3.5	Large Dataset	14
3.6	Data Preprocessing	14
3.6.1	Removing Unnecessary Features	15
3.6.2	Removing Duplicate Records	15
3.6.3	Handling Missing and Infinite Values	15
3.6.4	Feature Scaling	15
3.6.5	Categorical Encoding	15
3.6.6	Feature Selection	15
3.7	Anomaly Detection Methods	15
3.7.1	Multivariate Gaussian Model	16
3.7.2	Isolation Forest	16
3.7.3	One-Class Support Vector Machine	16
3.7.4	Deep Autoencoder	16
3.8	Supervised Classification Methods (UNSW-NB15)	16
3.9	Hyperparameter Optimization	17
3.10	Evaluation Metrics	17
3.11	Chapter Summary	17
4	Small Dataset Experiments	18
4.1	Introduction	18
4.2	Dataset Description	18
4.3	Data Preprocessing	18
4.4	Exploratory Data Analysis	19
4.5	Models Evaluated	19

4.6	Evaluation Metrics	19
4.7	Experimental Results	20
4.8	Discussion	20
4.9	Chapter Summary	21
5	Medium Dataset Experiments	22
5.1	Introduction	22
5.2	Part A – Supervised Classification on UNSW-NB15	22
5.2.1	Dataset Description	22
5.2.2	Data Preprocessing	23
5.2.3	Experimental Design	23
5.2.4	Models Evaluated	23
5.2.5	Experimental Results	24
5.2.6	Discussion	24
5.3	Part B – Unsupervised Anomaly Detection on CSE-CIC-IDS2018	25
5.3.1	Dataset Description	25
5.3.2	Data Preprocessing	25
5.3.3	Experimental Design	25
5.3.4	Models Evaluated	26
5.3.5	Experimental Results	27
5.3.6	Selection of the Autoencoder	27
5.3.7	Discussion	27
5.4	Comparing the Two Medium-Scale Studies	28
5.5	Chapter Summary	28
6	Large Dataset Experiments	29
6.1	Introduction	29
6.2	Dataset Description	29
6.3	Data Cleaning and Preprocessing	30
6.4	Construction of Training and Testing Datasets	30
6.5	Training Dataset Construction	31
6.5.1	Experiment A: Benign-only Training	31
6.5.2	Experiment B: Contaminated Training	31

6.6	Autoencoder Architecture	31
6.7	Hyperparameter Optimization	31
6.8	Evaluation Methodology	32
6.9	Experiment A: Autoencoder Trained Using Only Benign Traffic	32
6.9.1	Objective	32
6.9.2	Experimental Setup	32
6.9.3	Results on Different Testing Datasets	33
6.9.4	Analysis of Results	33
6.9.5	Advantages and Limitations	34
6.9.6	Discussion	34
6.10	Experiment B: Autoencoder Trained Using Contaminated Traffic	35
6.10.1	Objective	35
6.10.2	Experimental Setup	35
6.10.3	Training Dataset Composition	35
6.10.4	Experimental Results	36
6.10.5	Result Analysis	36
6.10.6	Advantages and Limitations	37
6.10.7	Discussion	37
6.11	Comparison Between Both Training Strategies	38
6.12	Chapter Summary	39
7	Conclusion and Future Work	40
7.1	Conclusion	40
7.2	Future Work	41

Chapter 1

Introduction

1.1 Introduction

With the rapid growth of digital communication and Internet-connected devices, computer networks have become an integral part of modern society. Governments, industries, educational institutions, financial organisations, healthcare systems and individuals depend heavily on networked systems for communication, information sharing and service delivery. While these advances have improved connectivity and productivity, they have also increased the number and sophistication of cyber threats. Attackers continuously develop new techniques to exploit network vulnerabilities, making network security one of the most important challenges of the digital era.

Network security aims to protect computer systems and communication infrastructure from unauthorised access, misuse, modification, and disruption. Firewalls, encryption, authentication systems, and Intrusion Detection Systems (IDS) are commonly used to safeguard network resources [1]. Among these, IDS play a critical role by continuously monitoring network activity and identifying behaviour that may indicate an attack [1].

Traditional intrusion detection relies mainly on signature matching, comparing incoming traffic against a database of previously known attack patterns [1]. This performs well against known attacks but cannot identify threats that have not been catalogued before. Because modern attacks frequently evolve to evade signature-based detection, anomaly detection techniques have attracted growing research attention [2, 11]: rather than matching signatures, they learn what normal behaviour looks like and flag deviations from it.

Machine learning provides an effective framework for anomaly detection because it can automatically learn statistical and behavioural patterns from large volumes of network traffic, without relying entirely on manually designed rules [12]. This makes it particularly

well suited to detecting zero-day attacks and other previously unseen threats [13].

This internship investigates machine learning and deep learning techniques for computer log anomaly detection across datasets of different scale. The work progresses systematically from a small, publicly available dataset to a large, realistic intrusion detection dataset, allowing the strengths and limitations of different anomaly detection approaches to be examined under increasingly demanding conditions.

1.2 Computer Logs

Computer logs are chronological records generated automatically by operating systems, applications, servers, databases, and network devices during normal operation [14]. They capture events describing system behaviour, user activity, application execution, and security-relevant occurrences.

Common examples include operating system logs, authentication logs, firewall logs, application logs, server logs, and network traffic logs. Security analysts routinely examine these logs to identify abnormal activity, investigate incidents, diagnose failures, and monitor network performance [14].

Because logs describe system behaviour over time, they are an excellent data source for anomaly detection. Rather than relying solely on predefined attack signatures, machine learning models can learn normal behavioural patterns directly from log data and flag unusual events that may represent attacks [2, 11].

1.3 Network Traffic Logs

Network traffic logs describe communication between devices on a network. Unlike system logs, they characterise data flows rather than individual system events. A typical network flow record contains:

- Source and destination IP address
- Source and destination port
- Communication protocol
- Flow duration

- Packet and byte counts
- Packet arrival times and other flow statistics

Rather than storing every transmitted packet, flow records summarise communication using statistical features. This reduces storage requirements while retaining enough behavioural information for machine learning. Benchmark datasets such as CSE-CIC-IDS2018 represent traffic using precisely this kind of flow-based feature set extracted from packet captures, making them well suited to anomaly detection research [7].

1.4 Network Anomaly Detection

Network anomaly detection identifies activity that deviates significantly from normal behaviour – deviations that may correspond to denial-of-service attacks, malware infections, unauthorised access, data exfiltration or insider threats [2].

Unlike signature-based detection, anomaly detection does not require prior knowledge of every attack type. Instead, it builds a model of normal behaviour and classifies observations that differ substantially from that model as anomalous [2]. Depending on the availability of labelled data, anomaly detection may be supervised, semi-supervised, or fully unsupervised [2, 12].

1.5 Motivation

Traditional intrusion detection struggles with previously unseen attacks [1]. Enterprise networks generate millions of flows daily, making manual inspection infeasible, and attack techniques continue to evolve, rendering static rule-based systems insufficient. Machine learning offers the ability to learn behavioural patterns from historical data and flag abnormal activity without explicit attack signatures [2, 11] – but choosing an appropriate model depends on dataset characteristics, feature quality, class imbalance, and computational cost. This internship investigates these factors systematically across datasets of increasing scale and complexity.

1.6 Problem Statement

Although many machine learning algorithms have been proposed for network anomaly detection, their performance varies considerably with dataset characteristics, preprocessing choices, feature engineering, and training strategy [11]. In practice, labelled attack data is scarce, which makes supervised learning difficult to apply at scale. There is therefore a need to systematically investigate unsupervised and semi-supervised methods that can learn normal network behaviour while maintaining high detection accuracy under varying attack frequencies [2].

1.7 Objectives

The primary objectives of this research project are as follows:

1. Study the fundamentals of machine learning for network anomaly detection.
2. Perform exploratory data analysis and preprocessing across multiple network traffic datasets.
3. Compare supervised classification with anomaly detection on a small labelled dataset.
4. Evaluate multiple anomaly detection algorithms on medium-scale network intrusion datasets, both supervised (UNSW-NB15) and unsupervised (CSE-CIC-IDS2018).
5. Identify the most effective anomaly detection model through comparative experimental analysis.
6. Investigate Deep Autoencoder performance under different training strategies on a large-scale dataset.
7. Analyse the robustness of anomaly detection under varying anomaly rates of 0.2%, 1%, and 5%.

1.8 Major Contributions

The major contributions of this work are as follows:

- Conducted a comparative study of supervised classification and anomaly detection methods on a small network traffic dataset.
- Performed extensive preprocessing and feature engineering for supervised classification on UNSW-NB15 [4], evaluating seven classifiers under four experimental configurations.
- Performed extensive preprocessing and feature engineering for unsupervised anomaly detection on CSE-CIC-IDS2018 [7].
- Evaluated four anomaly detection models on CSE-CIC-IDS2018 under multiple configurations combining feature selection and hyperparameter optimisation [6].
- Selected the Autoencoder for detailed investigation based on this comparative analysis.
- Investigated two Autoencoder training strategies – benign-only and contaminated training – at large scale.
- Evaluated the trained models on three independent test sets with different anomaly rates and analysed the effect of training data contamination on detection performance.

1.9 Datasets Used

Three publicly available datasets of increasing size were used in this internship: a small labelled dataset for initial experimentation, two medium-scale benchmark datasets for comparative model evaluation, and a large realistic dataset for the primary Autoencoder study.

1. **Small Dataset: Network Traffic Anomaly Detection Dataset [3].** Approximately 1,000 labelled network flow records were used for an initial comparison of supervised classification and anomaly detection methods.
2. **Medium Datasets: UNSW-NB15 and CSE-CIC-IDS2018.** Two independent benchmark intrusion detection datasets were used at the comparative-analysis stage.

- **UNSW-NB15** [4, 5]. Developed by the Australian Centre for Cyber Security (UNSW Canberra), UNSW-NB15 combines real modern normal traffic with synthetically generated attack traffic spanning nine attack categories, and was used here for supervised binary classification (Normal vs. Attack) across seven classifiers.
 - **CSE-CIC-IDS2018** (02-15-2018.csv) [15, 8]. Generated by the Canadian Institute for Cybersecurity, this file was used to compare four unsupervised anomaly detection algorithms under different experimental configurations.
3. **Large Dataset: CSE-CIC-IDS2018** (02-15-2018.csv) [15, 8]. The same source file was used at substantially larger scale for the primary Autoencoder study, comparing benign-only and contaminated training under three independent test anomaly rates (0.2%, 1%, 5%).

This progression – small, dual medium-scale, then large – enables a comprehensive analysis of machine learning and deep learning techniques for network anomaly detection under increasingly realistic conditions.

1.10 Report Organization

This report consists of seven chapters. Chapter 1 introduces the research problem, motivation, objectives, datasets, and organisation of the report. Chapter 2 reviews existing literature on machine learning and anomaly detection. Chapter 3 describes the anomaly detection techniques and evaluation metrics used throughout the work. Chapter 4 presents experiments on the small dataset. Chapter 5 presents the medium-scale comparative study, covering supervised classification on UNSW-NB15 and unsupervised anomaly detection on CSE-CIC-IDS2018, and explains why the Autoencoder was selected for further study. Chapter 6 presents the primary contribution of the internship: a comparison of benign-only and contaminated Autoencoder training on a large-scale dataset. Chapter 7 concludes the report, discusses limitations, and outlines directions for future work.

Chapter 2

Literature Review

2.1 Introduction

Network anomaly detection has become an important research area because of the growing dependence on computer networks for communication, cloud computing, financial transactions, healthcare, and industrial systems [2]. As traffic volumes grow, manual monitoring becomes impractical, motivating automated systems capable of identifying malicious behaviour in large volumes of traffic.

Traditional Intrusion Detection Systems (IDS) rely primarily on predefined signatures to identify known attacks [1]. While effective against previously observed attack patterns, such systems fail when attackers introduce new or modified strategies. Machine learning techniques address this limitation by learning patterns directly from data rather than relying solely on predefined rules [12, 11].

This chapter reviews the evolution of intrusion detection, discusses supervised and unsupervised learning approaches, summarises prior research on anomaly detection, and identifies the research gap that motivates the present work.

2.2 Intrusion Detection Systems

An IDS monitors network or system activity to identify malicious behaviour as early as possible so that defensive action can be taken [1]. IDS are generally classified into two categories.

- **Signature-based detection.** Incoming traffic is compared against a database of known attack signatures [1]; a match is classified as malicious. Such systems achieve high accuracy on known attacks with few false alarms, but cannot detect attacks for which no signature exists.

- **Anomaly-based detection.** The system first learns the characteristics of normal behaviour, then flags traffic that deviates significantly from that learned behaviour [2]. Anomaly-based detection can identify previously unseen attacks but tends to generate more false alarms, since unusual legitimate traffic can also appear anomalous.

Because modern attacks continuously evolve, anomaly-based detection using machine learning remains an active research area [11, 2].

2.3 Machine Learning for Anomaly Detection

Machine learning allows computers to identify patterns from historical data without being explicitly programmed for every case [12]. In cybersecurity, models learn relationships between traffic features and use them to distinguish normal from malicious behaviour. These techniques are broadly divided into supervised and unsupervised approaches [12, 2].

2.3.1 Supervised Learning

Supervised learning requires a labelled dataset in which every network flow is already classified as normal or malicious [12]. The algorithm learns the relationship between input features and labels during training, then predicts the class of unseen traffic. Popular algorithms include Logistic Regression [16], Decision Tree [17], Random Forest [18], Support Vector Machine [19], XGBoost [20] and LightGBM [21]. Supervised methods generally achieve high accuracy when sufficient labelled attack data is available, but collecting such data is expensive, since new attacks appear continuously.

2.3.2 Unsupervised Learning

Unsupervised anomaly detection does not require labelled attack samples; it learns only the characteristics of normal traffic and flags significant deviations as anomalous [2]. Common algorithms include Isolation Forest [22], Autoencoder [9], One-Class Support Vector Machine [23], Multivariate Gaussian [24], Gaussian Mixture Model [12] and Local Outlier Factor [2]. These approaches are attractive for cybersecurity precisely because labelled attack data is often unavailable or incomplete.

2.4 Deep Learning for Network Anomaly Detection

Deep learning has attracted significant attention for intrusion detection because it can automatically learn complex feature representations from large datasets [11], reducing the need for extensive manual feature engineering. Architectures applied to intrusion detection include Deep Neural Networks, Convolutional Neural Networks, Recurrent Neural Networks, LSTM networks and Autoencoders [9, 11]. Among these, Autoencoders are especially widely used for anomaly detection because they can be trained using only normal traffic [9].

2.5 Autoencoder-Based Anomaly Detection

An Autoencoder is a neural network trained to reconstruct its own input [9], consisting of an encoder and a decoder.

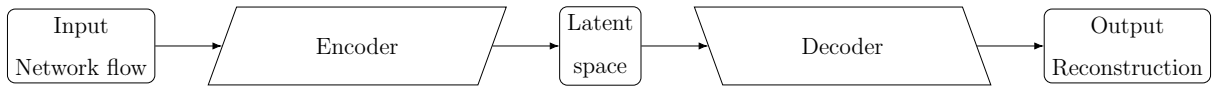


Figure 2.1: Autoencoder architecture: the encoder compresses the input into a lower-dimensional latent representation, and the decoder reconstructs the original input from this representation [9, 25].

The encoder compresses the input into a lower-dimensional latent representation; the decoder reconstructs the original input from this representation. Training minimises reconstruction error between input and output [9]. When trained only on normal traffic, the model reconstructs normal observations with small error, while anomalous traffic differing from the learned patterns – produces larger reconstruction error [2]. Comparing this error against a threshold identifies anomalous records. Because Autoencoders require no labelled attack samples during training, they are well suited to practical anomaly detection.

2.6 Publicly Available Intrusion Detection Datasets

Several benchmark datasets have been developed to evaluate intrusion detection algorithms under different network environments and attack scenarios, including KDD Cup 1999 [26], NSL-KDD [27], UNSW-NB15 [4] and CSE-CIC-IDS2018 [7]. Earlier datasets

such as KDD Cup 1999 and NSL-KDD have been studied extensively but no longer reflect modern network behaviour [27]. More recent datasets such as UNSW-NB15 and CSE-CIC-IDS2018 contain realistic traffic from modern network environments with a wider variety of attack scenarios, which is why both were selected for the medium-scale stage of this internship [4, 7].

2.7 Summary of Previous Research

Prior studies show that supervised algorithms achieve excellent performance when sufficient labelled attack data is available, with tree-based ensembles such as Random Forest [18], XGBoost [20] and LightGBM [21] consistently outperforming linear classifiers on nonlinear traffic relationships [1, 12]. Since labelled attack data is difficult to obtain in practice, many studies instead focus on unsupervised anomaly detection requiring only normal traffic during training [1, 11, 2]. Isolation Forest offers fast training and good scalability, while One-Class SVM performs well on smaller datasets but becomes expensive at scale [22, 23]. Deep Autoencoders often achieve superior performance by learning compact representations of normal traffic and detecting anomalies through reconstruction error [9, 11], though their performance depends heavily on preprocessing, feature selection, and hyperparameter optimisation [6, 10]. These observations directly motivated the experimental design of this internship.

2.8 Research Gap

Most prior work evaluates models on a single dataset or a single experimental setting; comparatively few studies examine performance across datasets of different sizes, or under multiple anomaly rates representative of real network conditions [2]. Many published comparisons also involve only a small number of algorithms, without systematically examining feature selection, hyperparameter optimisation and different training strategies together [6]. This motivates a systematic study that progresses from small to large datasets, compares multiple detection methods on more than one medium-scale benchmark, and analyses Autoencoder performance under varying anomaly rates and training-data compositions.

2.9 Chapter Summary

This chapter reviewed intrusion detection fundamentals, contrasted signature-based and anomaly-based detection, and discussed supervised and unsupervised learning, deep learning, Autoencoder-based detection, and commonly used benchmark datasets. The identified research gap motivates the experimental work in the remaining chapters, beginning with the datasets and methods used throughout this internship.

Chapter 3

Datasets and Anomaly Detection Methods

3.1 Introduction

The quality of an anomaly detection system depends not only on the algorithm but also on dataset quality, preprocessing, feature engineering, and evaluation methodology [2]. This chapter introduces the datasets used throughout the internship, the preprocessing pipeline applied to each, the anomaly detection methods investigated, and the evaluation metrics used for comparison.

3.2 Datasets Used

Three publicly available datasets were used, spanning small, medium, and large scale. The medium-scale stage uses *two* independent benchmark datasets – UNSW-NB15 [4, 5] and CSE-CIC-IDS2018 [7, 8] – to evaluate both supervised and unsupervised detection approaches on realistic traffic before committing to a large-scale study.

Table 3.1: Datasets used during the internship [3, 4, 5, 7, 15, 8]

Dataset	Scale	Records	Task	Role in Internship
Network Traffic Anomaly Detection Dataset	Small	~1,000	Binary classification	Baseline comparison of supervised and anomaly detection methods.
UNSW-NB15	Medium	257,673	Binary classification	Supervised classifier comparison with feature selection and Optuna tuning.
CSE-CIC-IDS2018 (02-15-2018.csv, medium subset)	Medium	1,048,575 (raw)	Unsupervised anomaly detection	Comparison of four anomaly detection algorithms.
CSE-CIC-IDS2018 (02-15-2018.csv, large subset)	Large	874,750 (cleaned)	Unsupervised anomaly detection	Detailed Autoencoder study under two training strategies.

Progressing from a small dataset toward large, realistic traffic was intentional: it allowed preprocessing steps, model behaviour, and evaluation metrics to be understood before tackling datasets with hundreds of thousands or millions of records.

3.3 Small Dataset

The Network Traffic Anomaly Detection Dataset (Kaggle) [3] contains approximately 1,000 network flow records with numerical features and a binary label (normal/anomalous). Its small size enabled rapid iteration across preprocessing strategies and models before moving to larger, more realistic data.

3.4 Medium Datasets

3.4.1 UNSW-NB15

UNSW-NB15 was developed by the Australian Centre for Cyber Security (UNSW Canberra) and combines real normal traffic with synthetically generated attack traffic across nine attack categories [4, 5]. The combined training and testing partitions used in this study contain 257,673 records and 45 original features (43 after removing identifier

and category columns), expanding to 197 features after one-hot encoding of categorical fields. The class distribution is moderately imbalanced toward attacks (63.91% Attack vs. 36.09% Normal), which is representative of the labelled-attack-category composition of this particular benchmark. UNSW-NB15 was used exclusively for supervised binary classification in this work.

3.4.2 CSE-CIC-IDS2018 (Medium Subset)

The 02-15-2018.csv file of CSE-CIC-IDS2018, produced by the Canadian Institute for Cybersecurity [7, 8], contains over one million flow records with roughly eighty numerical features describing packet and byte counts, flow duration, inter-arrival time, protocol information, and TCP flag statistics. Only the Benign and DoS-GoldenEye classes were retained, simplifying the problem to binary anomaly detection; the DoS-Slowloris class was removed. This dataset was used exclusively for unsupervised anomaly detection, since it more closely reflects the practical scenario in which labelled attack data is unavailable [2].

Using two independent medium-scale datasets – one supervised, one unsupervised – allowed both learning paradigms to be evaluated under realistic, larger-than-toy conditions before selecting a single model family for the large-scale study.

3.5 Large Dataset

The large-scale study reuses the same CSE-CIC-IDS2018 source file [7, 8], but with substantially more benign training data (up to 783,444 records) and three independent test sets at anomaly rates of 0.2%, 1%, and 5%, allowing detection performance to be examined under progressively more challenging class imbalance.

3.6 Data Preprocessing

Raw network traffic cannot be used directly for machine learning. The following preprocessing stages were applied, adapted as needed to each dataset.

3.6.1 Removing Unnecessary Features

Columns that do not describe intrinsic traffic behaviour – most notably timestamps – were removed prior to training.

3.6.2 Removing Duplicate Records

Exact duplicate rows were removed to prevent repeated traffic patterns from biasing model training.

3.6.3 Handling Missing and Infinite Values

Missing values were imputed using the median, which is less sensitive to outliers than the mean. Infinite values – arising in features such as Flow Bytes/s and Flow Packets/s from division by a zero-duration flow – were first converted to missing values and then imputed identically.

3.6.4 Feature Scaling

Because feature magnitudes vary substantially (packet counts vs. flag counters, for example), scaling was applied before training. `StandardScaler`, `MinMaxScaler`, and `RobustScaler` [28] were used depending on the assumptions of each downstream algorithm.

3.6.5 Categorical Encoding

For UNSW-NB15, the categorical fields `proto`, `service`, and `state` were one-hot encoded prior to scaling.

3.6.6 Feature Selection

Mutual Information [10] and Variance Threshold [28] filtering were used to identify the most informative features, reducing dimensionality and computational cost while aiming to improve generalisation.

3.7 Anomaly Detection Methods

Four unsupervised anomaly detection methods were investigated on CSE-CIC-IDS2018, each identifying anomalies using a different principle.

3.7.1 Multivariate Gaussian Model

Assumes normal traffic follows a multivariate normal distribution parameterised by a mean vector and covariance matrix estimated from normal samples. Mahalanobis distance [24] from this distribution is used as the anomaly score. Computationally efficient, but reliant on an approximately Gaussian feature distribution.

3.7.2 Isolation Forest

An ensemble method that isolates observations through recursive random partitioning of the feature space [22]. Anomalies typically require fewer partitions to isolate, and the average path length across trees forms the anomaly score. Efficient and scalable to large datasets.

3.7.3 One-Class Support Vector Machine

Learns a boundary enclosing normal observations [23]; traffic outside this boundary is flagged as anomalous. Effective for compact normal distributions but computationally expensive at scale.

3.7.4 Deep Autoencoder

Learns a compressed, nonlinear representation of normal traffic by minimising reconstruction error [9]; large reconstruction error at inference time indicates an anomaly. Able to model far more complex feature relationships than the statistical alternatives above.

3.8 Supervised Classification Methods (UNSW-NB15)

For UNSW-NB15, seven supervised classifiers were evaluated: Logistic Regression, K-Nearest Neighbours, Linear Support Vector Machine, Random Forest, XGBoost, LightGBM and a Voting Classifier combining the strongest individual models. These represent a spectrum from simple linear decision boundaries to high-capacity tree ensembles, allowing the effect of model complexity on classification performance to be examined directly.

3.9 Hyperparameter Optimization

Optuna [6] was used throughout this work for automatic hyperparameter optimisation, searching parameter combinations to maximise validation performance. For the Autoencoder, the optimised parameters included encoding dimension, hidden layer size, number of hidden layers, learning rate, dropout rate and L2 regularisation [9, 25]. Analogous tuning was applied to the tree-based classifiers used on UNSW-NB15 (e.g. tree depth, learning rate, number of estimators).

3.10 Evaluation Metrics

Because anomaly detection datasets are highly imbalanced, accuracy alone is insufficient [29]. Accuracy, Precision, Recall, F1-score, False Positive Rate (FPR) and False Negative Rate (FNR) were therefore all considered [29], with F1-score treated as the primary metric since it balances Precision and Recall. FPR and FNR were analysed alongside F1 because both missed attacks and excessive false alarms carry significant practical costs in an operational intrusion detection system.

3.11 Chapter Summary

This chapter introduced the datasets used during the work – including the two independent medium-scale benchmarks, UNSW-NB15 and CSE-CIC-IDS2018 – and described the shared preprocessing pipeline, the anomaly detection and classification methods investigated, and the evaluation metrics used for comparison. The next chapter presents the first experimental stage, on the small dataset.

Chapter 4

Small Dataset Experiments

4.1 Introduction

The first stage of the work used a small, publicly available network traffic dataset [3] to build practical experience in preprocessing, exploratory analysis, feature engineering, model training, and evaluation, before moving to datasets containing hundreds of thousands or millions of records. A key question at this stage was not only which model performed best, but whether supervised classification or unsupervised anomaly detection was more suitable given the available data.

4.2 Dataset Description

The dataset used was the Network Traffic Anomaly Detection Dataset (Kaggle) [3], containing approximately 1,000 records with numerical and Boolean features and a binary label (normal/anomalous). Boolean features carried little discriminative information for the models used here and were removed, leaving only numerical features. Class distribution was imbalanced: approximately 90% normal traffic and 10% anomalous traffic, consistent with the rarity of attacks typically observed in cybersecurity data [2].

4.3 Data Preprocessing

The dataset was checked for missing values, duplicates, and constant features; no missing values or duplicates were found, but one feature (`mean_packet_size`) was constant across all records and was removed. Remaining numerical features were standardised using `StandardScaler` [28]. Since the dataset was imbalanced, the Synthetic Minority Oversampling Technique (SMOTE) [30] was applied to generate synthetic minority-class samples, producing an equal number of normal and anomalous samples (1,800 total: 1,440

training, 360 testing after an 80:20 stratified split).

4.4 Exploratory Data Analysis

Histograms and boxplots were used to examine feature distributions and identify outliers, particularly in `src_port`, `dst_port`, and `inter_arrival_time`. A correlation matrix was constructed to identify redundant features. Comparing feature averages between normal and anomalous traffic showed noticeable differences in several variables – including `packet_count_5s`, `inter_arrival_time`, and `spectral_entropy` – giving confidence that the dataset contained learnable structure.

4.5 Models Evaluated

Two categories of algorithm were compared. Supervised classifiers, trained on the SMOTE-balanced dataset, included Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, and XGBoost. Unsupervised anomaly detection methods, trained only on normal observations, included Isolation Forest, Gaussian Mixture Model, Hidden Markov Model, Deep Autoencoder and Elliptic Envelope. The complete source code for the small dataset experiments is publicly available on GitHub [\[31\]](#).

4.6 Evaluation Metrics

Accuracy, Precision, Recall, and F1-score were computed for every model [\[29\]](#), with F1-score treated as the primary indicator; confusion matrices were also examined to characterise the distribution of true/false positives and negatives.

4.7 Experimental Results

Table 4.1: Performance comparison of machine learning and anomaly detection models on the small dataset

Model	Accuracy	Precision	Recall	F1-Score
XGBoost	0.9306	0.9500	0.9100	0.9300
Random Forest	0.9194	0.9415	0.8944	0.9174
Decision Tree	0.8389	0.8427	0.8333	0.8380
Support Vector Machine	0.6000	0.5928	0.6389	0.6150
Logistic Regression	0.6222	0.6310	0.5889	0.6088
Gaussian Mixture Model	0.5333	0.6429	0.1500	0.2432
Hidden Markov Model	0.4750	0.3333	0.0500	0.0870
Isolation Forest	0.4222	0.2200	0.0611	0.0956
Autoencoder	0.4972	0.4615	0.0333	0.0621
Elliptic Envelope	0.4694	0.1765	0.0167	0.0305

XGBoost achieved the strongest overall performance, closely followed by Random Forest; Decision Tree trailed both ensemble methods, while Logistic Regression and SVM – both linear – performed noticeably worse, suggesting that normal/anomalous traffic is not linearly separable even on this small dataset. Every unsupervised anomaly detection method (Autoencoder, Isolation Forest, Hidden Markov Model, Gaussian Mixture Model, Elliptic Envelope) scored well below the supervised classifiers.

4.8 Discussion

Supervised learning clearly outperformed anomaly detection on this dataset, unsurprisingly, since labelled attacks were directly available during training. Tree-based ensembles outperformed linear models, indicating nonlinear structure in the feature relationships. The weak performance of unsupervised methods trained only on normal traffic suggests that the anomalies in this small dataset were not sufficiently distinct from normal traffic

for reconstruction- or density-based methods to separate cleanly [2]. Despite lower accuracy here, unsupervised methods remain attractive in practice precisely because they do not require labelled attacks – a limitation of the small dataset that motivated the transition to larger, more realistic benchmarks.

4.9 Chapter Summary

Supervised ensemble methods clearly outperformed anomaly detection methods on the small dataset, but the latter’s independence from labelled attack data motivates their continued study at larger scale. The next chapter presents the medium-scale comparative study across two independent benchmark datasets: UNSW-NB15 (supervised) and CSE-CIC-IDS2018 (unsupervised).

Chapter 5

Medium Dataset Experiments

5.1 Introduction

The second stage of the work moved to medium-scale, realistic network traffic. Two independent benchmark datasets were used, each addressing a different learning paradigm: UNSW-NB15 [4] for supervised classification, where labelled attack categories are directly available and CSE-CIC-IDS2018 [7] for unsupervised anomaly detection, which better reflects deployments where labelled attacks cannot be assumed. Studying both in parallel makes it possible to compare paradigms directly under comparable experimental rigour – four configurations combining feature selection and hyperparameter optimisation in both cases – before committing to a single model family for the large-scale study in Chapter 6.

5.2 Part A – Supervised Classification on UNSW-NB15

5.2.1 Dataset Description

UNSW-NB15 [4, 5] combines real normal traffic with synthetic attack traffic across nine attack categories. The combined training and testing partitions used here contain 257,673 records with 45 original features. After removing the `id` and `attack_cat` columns and one-hot encoding the categorical fields `proto`, `service`, and `state` [28], the feature space expands to 197 dimensions. The binary class distribution is 93,000 Normal (36.09%) and 164,673 Attack (63.91%) records.

5.2.2 Data Preprocessing

Training and testing partitions were combined prior to preprocessing. The `id` and `attack_cat` columns were removed, and one-hot encoding was applied to `proto`, `service`, and `state`. `StandardScaler` [28] was applied to all numerical features, and a binary label (Normal/Attack) was constructed for classification.

5.2.3 Experimental Design

Four experimental configurations were evaluated for every classifier, identical in structure to the design used later for CSE-CIC-IDS2018, enabling a like-for-like comparison of how feature selection and hyperparameter tuning affect supervised and unsupervised approaches respectively.

Table 5.1: Experimental Configurations Evaluated (UNSW-NB15 and CSE-CIC-IDS2018) [10, 6]

Exp	Description	Feature Selection	HPO
1	Baseline model, all features	No	No
2	Model with selected features only	Yes	No
3	Baseline model with Optuna optimisation	No	Yes
4	Selected features with Optuna optimisation	Yes	Yes

Mutual Information feature selection [10] was applied in Experiments 2 and 4, and Optuna Hyperparameter Optimization (HPO) [6] (20 trials) was conducted in Experiments 3 and 4.

5.2.4 Models Evaluated

Seven classifiers were compared: Logistic Regression, K-Nearest Neighbours, Linear Support Vector Machine, Random Forest, XGBoost, LightGBM, and a Voting Classifier ensembling the top-performing models. The complete implementation for the UNSW-NB15 dataset experiments is publicly available on GitHub [32].

5.2.5 Experimental Results

Table 5.2: Best Configuration per Model – UNSW-NB15 Supervised Classification

Model	Configuration	Accuracy	Precision	Recall	F1
XGBoost	Optuna – All Features	0.9214	0.9225	0.8982	0.9102
Random Forest	Optuna – All Features	0.9207	0.9177	0.9022	0.9099
Voting Classifier	Optuna – Selected Features	0.9194	0.9052	0.9143	0.9097
LightGBM	Optuna – Selected Features	0.9202	0.9238	0.8938	0.9086
KNN	Optuna – All Features	0.8883	0.8822	0.8637	0.8728
Logistic Regression	Optuna – All Features	0.8480	0.7746	0.9270	0.8440
Linear SVM	Initial	0.8458	0.7636	0.9450	0.8447

XGBoost achieved the highest overall accuracy and F1-score (92.14% / 91.02%), closely followed by Random Forest, the Voting Classifier, and LightGBM – all within roughly half a percentage point of one another after Optuna optimisation. LightGBM achieved the highest Precision (92.38%), and the Voting Classifier the highest Recall among the ensemble models. KNN improved substantially with tuning but remained behind the ensembles; Logistic Regression and Linear SVM achieved high Recall but comparatively low Precision, reflecting their linear decision boundary and a corresponding tendency toward false positives.

5.2.6 Discussion

As on the small dataset, tree-based ensembles clearly outperformed linear classifiers on UNSW-NB15, confirming that the relationship between traffic features and attack labels is substantially nonlinear. HPO [6] improved almost every model, with Random Forest showing the largest gain; feature selection reduced dimensionality with only marginal effect on predictive performance, indicating that a sizeable share of the 197 encoded features is redundant. These results establish a strong supervised baseline and confirm that, when labelled attacks are available, ensemble classifiers are highly effective on realistic intrusion traffic.

5.3 Part B – Unsupervised Anomaly Detection on CSE-CIC-IDS2018

5.3.1 Dataset Description

The 02-15-2018.csv file of CSE-CIC-IDS2018 [7, 8] contains over one million flow records with approximately eighty numerical features. Only Benign and DoS-GoldenEye traffic were retained (DoS-Slowloris was removed), framing the problem as binary anomaly detection.

5.3.2 Data Preprocessing

The Timestamp column was removed, duplicate records were dropped, and missing and infinite values (concentrated in Flow Bytes/s and Flow Packets/s) were imputed using the median. Depending on each algorithm’s assumptions, features were scaled using StandardScaler, RobustScaler, or MinMaxScaler [28]; strongly skewed features were transformed using the Yeo-Johnson transformation [33] where appropriate.

5.3.3 Experimental Design

A training set of 100,000 benign records was used to fit each model. Three independent test sets of 30,000 records each were constructed with hierarchical, nested anomaly rates of 0.2%, 1%, and 5%, so that a test set at a higher anomaly rate is a strict superset of the lower-rate test sets in its attack composition.

Table 5.3: Composition of Training and Testing Datasets – CSE-CIC-IDS2018 (Medium Subset) [7, 8]

Dataset	Total	Benign	Attack	Rate	Levels	Purpose
Training Set	100,000	100,000	0	0%	None	Train models using only benign traffic.
Test Set 1	30,000	29,940	60	0.2%	L1	Very low anomaly contamination.
Test Set 2	30,000	29,700	300	1%	L1+L2	Low anomaly contamination.
Test Set 3	30,000	28,500	1,500	5%	L1+L2+L3	Higher anomaly contamination.

The same four experimental configurations described in Table 5.1 were applied here, using Mutual Information / Variance Threshold feature selection [10, 28] (Experiments 2 and 4) and Optuna HPO [6] (Experiments 3 and 4). Across four algorithms, four configurations, and three test sets, this design yields 48 individual evaluations.

5.3.4 Models Evaluated

Four unsupervised methods were compared: Multivariate Gaussian (MVN), Isolation Forest, One-Class SVM (OCSVM) and Deep Autoencoder, each described in Section 3.6. The complete implementation for the CSE-CIC-IDS2018 dataset experiments is publicly available on GitHub [34].

5.3.5 Experimental Results

Table 5.4: Overall Best Configuration for Each Model – CSE-CIC-IDS2018 (Medium Subset)

Model	Exp	Test Set	F1	Precision	Recall	FPR	FNR
Autoencoder	3	5%	0.9298	0.8807	0.9847	0.0070	0.0153
OCSVM	4	5%	0.8033	0.7254	0.9000	0.0179	0.1000
Isolation Forest	4	5%	0.7319	0.6874	0.7827	0.0187	0.2173
MVN	2	5%	0.4291	0.6743	0.3147	0.0080	0.6853

Hyperparameter optimisation consistently improved every model, with a substantially larger effect than feature selection alone. The Autoencoder achieved the highest average F1-score across all configurations and test sets, followed by Isolation Forest, OCSVM, and MVN. Isolation Forest showed the largest relative gain from optimisation, improving from an unoptimised F1 of roughly 0.63 to 0.85 once Optuna-tuned. MVN remained the most stable model across configurations but the weakest overall performer, consistent with its restrictive Gaussian assumption.

5.3.6 Selection of the Autoencoder

The Autoencoder was carried forward to the large-scale study for four reasons: it achieved the highest average F1-score across all configurations; its performance remained comparatively stable across anomaly rates; it demonstrated the greatest capacity to model nonlinear feature relationships among the four candidates [9]; and it showed the largest absolute improvement after hyperparameter optimisation, suggesting further headroom for gains at larger scale. Isolation Forest was a competitive second choice, but the Autoencoder was consistently stronger across settings.

5.3.7 Discussion

No single anomaly detection algorithm performed optimally under every condition. MVN is computationally cheap but too restrictive for complex traffic distributions; OCSVM is competitive but expensive to scale; Isolation Forest scales well and responds strongly to

tuning. The Autoencoder achieved the best overall balance of accuracy, robustness, and scalability, justifying its selection as the sole focus of the large-scale study in Chapter 6.

5.4 Comparing the Two Medium-Scale Studies

Although UNSW-NB15 and CSE-CIC-IDS2018 address different learning paradigms, both studies point to the same underlying conclusion: tree-based and representation-learning models that can capture nonlinear structure (XGBoost/Random Forest on UNSW-NB15; Autoencoder on CSE-CIC-IDS2018) consistently outperform simpler linear or distribution-based alternatives, and hyperparameter optimisation [6] provides a larger performance gain than feature selection in both settings. The key practical difference is data availability: UNSW-NB15’s fully labelled attack categories make supervised classification directly applicable, whereas CSE-CIC-IDS2018 was deliberately treated as unsupervised to reflect deployments where labelled attacks cannot be assumed [2]. Because real-world intrusion detection more often resembles the latter scenario, the Autoencoder – rather than a supervised classifier – was selected as the basis for the large-scale investigation that follows.

5.5 Chapter Summary

This chapter presented two independent medium-scale studies: supervised classification on UNSW-NB15, where XGBoost, Random Forest, and a Voting Classifier all achieved F1-scores above 0.90, and unsupervised anomaly detection on CSE-CIC-IDS2018, where the Autoencoder achieved the highest F1-score (0.93) among four candidate algorithms. Both studies confirmed that hyperparameter optimisation and nonlinear modelling capacity are the dominant drivers of performance. The Autoencoder was selected for the large-scale study presented in the next chapter.

Chapter 6

Large Dataset Experiments

6.1 Introduction

This chapter presents the primary contribution of the internship. Building on the Autoencoder’s strong performance in Chapter 5, this chapter investigates its behaviour in detail on a much larger subset of CSE-CIC-IDS2018 [7], focusing not on comparing algorithms but on how training-data composition affects a single model. Two training strategies are compared: training exclusively on benign traffic, and training on data deliberately contaminated with a small proportion of attacks. Both are evaluated against three independent test sets with anomaly rates of 0.2%, 1%, and 5%, allowing the robustness of each strategy to be assessed as deployment conditions vary.

6.2 Dataset Description

The experiments use the CSE-CIC-IDS2018 traffic captured on 15 February 2018 [7, 8], retaining only Benign and DoS-GoldenEye traffic (Slowloris removed) to keep the study focused on a single, well-understood attack family without additional variability from multiple attack types. The raw file contains 1,048,575 records and 80 attributes.

6.3 Data Cleaning and Preprocessing

Table 6.1: Data Cleaning Pipeline Summary [7]

Step	Action	Rows Remaining	Rows Removed / Affected
1	Raw dataset loaded	1,048,575	–
2	Timestamp column dropped	1,048,575	1 column removed
3	Removed DoS attacks-Slowloris class	1,037,585	10,990 rows removed
4	Removed exact duplicate rows	874,750	162,835 rows removed (15.69%)
5	Imputed missing Flow Byts/s (label-wise median)	874,750	4,759 rows affected (0.544%), all Benign
6	Replaced infinite Flow Byts/s / Flow Pkts/s (label-wise median)	874,750	7,712 rows affected, all Benign

All numerical features were subsequently normalised using Min-Max Scaling [28], appropriate for the neural-network-based Autoencoder and ensuring stable gradient updates during training.

6.4 Construction of Training and Testing Datasets

Table 6.2: Composition of Testing Datasets – Large-Scale Study [7, 8]

Anomaly Rate	Benign	Attack	Total	Attack Composition
0.2%	49,900	100	50,000	Level 1
1%	49,500	500	50,000	Level 1 + Level 2
5%	47,500	2,500	50,000	Level 1 + Level 2 + Level 3

These three fixed test sets were used, unchanged, throughout every experiment in this chapter, with no overlap with any training data, to ensure a fair comparison between training strategies.

6.5 Training Dataset Construction

6.5.1 Experiment A: Benign-only Training

The Autoencoder is trained using 783,444 benign-only records, following the conventional assumption that attack samples are unavailable at training time [2, 9].

6.5.2 Experiment B: Contaminated Training

The Autoencoder is instead trained on data intentionally contaminated with a small proportion of attack samples, reflecting practical deployments where perfectly clean training data cannot be guaranteed.

6.6 Autoencoder Architecture

The Autoencoder comprises an encoder that progressively compresses the input into a lower-dimensional latent representation, and a decoder that reconstructs the original input from it [9]. Training minimises the Mean Squared Error between input and reconstruction using the Adam optimiser. Batch Normalization and Dropout layers were used to improve generalisation and reduce overfitting [9, 25]; normal traffic, well represented during training, produces low reconstruction error, while abnormal traffic produces larger error, forming the basis of the anomaly score [2].

6.7 Hyperparameter Optimization

Two architecture variants were evaluated: a default architecture and one tuned with Optuna [6] over encoding dimension, hidden layer size, number of hidden layers, learning rate, dropout rate, and L2 regularisation. The optimisation objective maximised F1-score while penalising both false positives and false negatives.

6.8 Evaluation Methodology

Each trained Autoencoder was evaluated on all three test sets using Precision, Recall, F1-score, FPR, and FNR, with F1-score as the primary criterion given the class imbalance inherent to all three test sets.

6.9 Experiment A: Autoencoder Trained Using Only Benign Traffic

6.9.1 Objective

This experiment follows the conventional anomaly detection assumption that only benign traffic is available for training [2]. Any flow the trained model cannot reconstruct accurately is flagged as anomalous, with reconstruction error above a calibrated threshold used as the decision rule.

6.9.2 Experimental Setup

To evaluate the performance of the Autoencoder trained exclusively on normal traffic, the following sequential pipeline was implemented:

1. Load the cleaned CSE-CIC-IDS2018 dataset.
2. Extract only benign traffic for training.
3. Apply feature preprocessing and normalisation.
4. Train the Autoencoder to convergence.
5. Compute reconstruction error for every test sample.
6. Calibrate the anomaly threshold on a held-out calibration set.
7. Evaluate on all three test sets.

The complete implementation for the benign-only large-scale Autoencoder experiments is publicly available on GitHub [35].

6.9.3 Results on Different Testing Datasets

Table 6.3: Autoencoder Performance Across Experimental Configurations – Benign-only Training

Exp	Test Set	Precision	Recall	F1	FPR	FNR
1	0.2%	0.1279	0.4400	0.1982	0.0060	0.5600
1	1%	0.3912	0.7840	0.5220	0.0123	0.2160
1	5%	0.6764	0.9968	0.8060	0.0251	0.0032
2	0.2%	0.0500	0.2000	0.0800	0.0076	0.8000
2	1%	0.3668	1.0000	0.5368	0.0174	0.0000
2	5%	0.7604	0.9612	0.8491	0.0159	0.0388
3	0.2%	0.3590	0.9800	0.5255	0.0035	0.0200
3	1%	0.7294	0.9220	0.8145	0.0035	0.0780
3	5%	0.9225	1.0000	0.9597	0.0044	0.0000
4	0.2%	0.6782	0.5900	0.6310	0.0006	0.4100
4	1%	0.8973	0.6640	0.7632	0.0008	0.3360
4	5%	0.9766	0.6676	0.7931	0.0008	0.3324

Table 6.4: Best Performance Configuration for Each Experiment – Benign-only Training

Exp	Best Test Set	Precision	Recall	F1	FPR	FNR
1	5%	0.6764	0.9968	0.8060	0.0251	0.0032
2	5%	0.7604	0.9612	0.8491	0.0159	0.0388
3	5%	0.9225	1.0000	0.9597	0.0044	0.0000
4	5%	0.9766	0.6676	0.7931	0.0008	0.3324

6.9.4 Analysis of Results

The Autoencoder learns normal traffic characteristics without requiring attack samples [9]: benign reconstruction error remains small throughout, while attack traffic produces

markedly larger error. Performance is, however, sensitive to the test set’s anomaly rate: at 0.2%, even a few misclassifications sharply reduce F1, since positives are extremely rare; as the anomaly rate rises to 5%, class separation becomes more apparent and Precision, Recall, and F1 all improve, an expected effect of larger, more stable minority-class samples.

6.9.5 Advantages and Limitations

Table 6.5: Benign-only Training: Advantages and Limitations

Advantages	Limitations
No labelled attack samples required for training.	Selecting an appropriate anomaly threshold remains difficult.
Closely resembles practical deployments where labelled attacks are unavailable.	Small threshold changes can materially shift Precision and Recall.
Learns a compact representation of normal behaviour.	Extremely rare attacks (0.2%) remain challenging to detect consistently.
Previously unseen attack types can still trigger large reconstruction error.	Highly variable benign traffic can itself inflate reconstruction error, producing false alarms.
Computationally efficient – only one class used during training.	Performance depends heavily on how representative the benign training sample is.

6.9.6 Discussion

A benign-only Autoencoder identifies anomalous traffic effectively without labelled attack data, which is valuable given how unrealistic it is to expect representative attack samples for every possible threat [2]. Detection difficulty scales inversely with anomaly rate, motivating the contaminated-training experiment that follows: does deliberately including a small share of attacks during training change this picture, and does it improve robustness to imperfect data collection?

6.10 Experiment B: Autoencoder Trained Using Contaminated Traffic

6.10.1 Objective

Perfectly clean training data is rarely available in practice: large organisational networks generate enormous traffic volumes that cannot be manually verified, so training sets may unintentionally include a small fraction of attack traffic. This experiment trains the Autoencoder on such contaminated data – without providing attack labels – to test whether the learned representation of normal behaviour remains sufficiently discriminative.

6.10.2 Experimental Setup

The procedure is identical to Experiment A except for the composition of the training data, which now contains a controlled proportion of attack records alongside benign traffic, using the same preprocessing pipeline, scaling, and architecture as Experiment A so that any performance difference can be attributed to training data composition alone. The complete implementation for the contaminated-training Autoencoder experiments is publicly available on GitHub [36].

6.10.3 Training Dataset Composition

Table 6.6: Composition of Contaminated Training Datasets [7, 8]

Contamination Level	Benign	Attack	Total	Attack Composition
0.2%	776,564	1,556	778,120	Level 1
1%	770,339	7,781	778,120	Level 1 + Level 2
5%	739,214	38,906	778,120	Level 1 + Level 2 + Level 3

6.10.4 Experimental Results

Table 6.7: Best Train–Test Combination for Each Experiment – Contaminated Training

Exp	Best Train → Test Combination	Precision	Recall	F1	FPR	FNR
1	Train 5% → Test 5% (Matched)	0.7728	0.9768	0.8629	0.0151	0.0232
2	Train 0.2% → Test 5% (Cross-domain)	0.8915	0.9664	0.9274	0.0062	0.0336
3	Train 1% → Test 5% (Cross-domain)	0.8911	0.9660	0.9271	0.0062	0.0340
4	Train 1% → Test 5% (Cross-domain)	0.6852	0.9352	0.7909	0.0226	0.0648

6.10.5 Result Analysis

Introducing attack records into training narrows the gap between benign and attack reconstruction error, since the network also partially learns to reconstruct anomalous samples. The effect scales with contamination level: at very low contamination, benign records still overwhelmingly dominate training, and reconstruction error for attacks remains distinctly higher than for benign traffic; as contamination rises, the model becomes progressively better at reconstructing attacks too, narrowing the separation that anomaly detection depends on. This behaviour is consistent with the theoretical expectation for reconstruction-based detection under label noise [2, 9].

6.10.6 Advantages and Limitations

Table 6.8: Contaminated Training: Advantages and Limitations

Advantages	Limitations
Better reflects realistic network environments.	High contamination levels reduce reconstruction-error separation.
Does not require a perfectly clean training set.	Excessive attack presence can be partially absorbed into the learned normal representation.
Less sensitive to minor contamination introduced during data collection.	Detection performance depends strongly on the contamination percentage.
Provides direct evidence of robustness under imperfect training conditions.	Threshold selection becomes harder as error distributions increasingly overlap.

6.10.7 Discussion

Because the Autoencoder does not learn explicit class boundaries, it reconstructs whatever it is shown often enough – so if attack samples become sufficiently frequent during training, the network gradually learns to reconstruct them too [9]. Encouragingly, the results show that *limited* contamination does not cause immediate performance collapse: benign traffic still dominates the training set numerically, so the dominant learned representation remains that of normal behaviour. This is a practically important finding, since perfectly clean training data is rarely available in operational cybersecurity settings [2].

6.11 Comparison Between Both Training Strategies

Table 6.9: Comparison of Both Training Strategies

Criterion	Benign-only Training	Contaminated Training
Training data	Only benign	Benign + attack
Requires perfectly clean dataset	Yes	No
Practical deployment suitability	Moderate	High
Ability to learn normal behaviour	Very high	High
Tolerance to training contamination	Low	High
Risk of learning attack patterns	Very low	Moderate
Overall detection performance	Excellent	Very good
Best average F1-score	0.9597	0.9271
Best performing scenario	Exp 3, 5% test set	Exp 2, Train 0.2% → Test 5%

Benign-only training produces a cleaner representation of normal behaviour and yields the strongest overall anomaly separation, but depends on curated, attack-free training data that is not always available. Contaminated training better reflects practical deployment conditions and remains highly effective under limited contamination, though excessive contamination measurably erodes detection performance. Both results reinforce a single conclusion: the quality and composition of the training set play a decisive role in reconstruction-based anomaly detection [2, 9].

6.12 Chapter Summary

This chapter presented the internship’s primary contribution: a detailed comparison of Autoencoder training strategies at scale. Benign-only training produced the strongest overall separation between normal and attack traffic (best $F1 = 0.9597$), while contaminated training remained highly effective (best $F1 = 0.9271$) and offers a more realistic alternative when perfectly clean data cannot be guaranteed. Both confirm that training data quality and composition materially influence reconstruction-based anomaly detection, and that Autoencoders remain robust detectors of network intrusions across increasingly demanding class-imbalance conditions.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This Summer Research Internship investigated machine learning and deep learning approaches to network anomaly detection across datasets of increasing scale and realism: a small labelled dataset, two independent medium-scale benchmarks (UNSW-NB15 and CSE-CIC-IDS2018), and a large-scale Autoencoder study.

On the small dataset, supervised ensembles (XGBoost, Random Forest) substantially outperformed unsupervised anomaly detection, reflecting the direct advantage of labelled attack data at small scale. The medium-scale stage extended this comparison in two directions simultaneously: on UNSW-NB15, supervised classifiers again performed strongly, with XGBoost, Random Forest, and a Voting Classifier all exceeding an F1-score of 0.90; on CSE-CIC-IDS2018, four unsupervised anomaly detection algorithms were compared under an identical experimental protocol, with the Autoencoder achieving the best overall F1-score (0.93) and motivating its selection for the large-scale study.

The large-scale experiments, focused exclusively on the Autoencoder, compared benign-only and contaminated training regimes across three test anomaly rates. Benign-only training achieved the strongest overall separation between normal and attack traffic, while contaminated training remained highly effective and more realistically reflects deployments where perfectly clean training data cannot be guaranteed.

Across all stages, hyperparameter optimisation via Optuna [6] produced substantial, consistent performance gains, and feature selection reduced model complexity while broadly preserving predictive performance. The quality and composition of training data emerged as a decisive factor in reconstruction-based anomaly detection specifically [2]. Overall, this internship demonstrates that Deep Autoencoders are an effective, practical solution for network anomaly detection, particularly in settings where labelled attack

data is limited or unavailable, while also showing that supervised ensembles remain highly competitive whenever labelled data, such as that in UNSW-NB15, is available.

7.2 Future Work

The following directions can be explored for future work:

- Extend the proposed framework to additional intrusion detection datasets, such as CICIDS2017 and TON_IoT.
- Investigate multiple attack categories jointly rather than focusing on a single attack family (DoS-GoldenEye) as in the large-scale study.
- Explore advanced architectures, including Variational Autoencoders, Transformer-based Autoencoders, Graph Neural Networks, and contrastive learning approaches.
- Develop adaptive threshold selection methods that adjust automatically to changing traffic distributions.
- Investigate online and incremental learning for continuous adaptation to evolving network behaviour.
- Evaluate the proposed models in real-time monitoring settings to assess computational efficiency and deployment feasibility.
- Apply explainable AI (XAI) techniques to improve the interpretability of anomaly detection decisions.
- Study federated anomaly detection frameworks that allow multiple organisations to collaboratively train intrusion detection models without sharing sensitive traffic.

These directions offer promising opportunities to further improve the accuracy, robustness, scalability, and practical deployability of machine learning-based intrusion detection systems.

Bibliography

- [1] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, “Survey of intrusion detection systems: Techniques, datasets and challenges,” *Cybersecurity*, vol. 2, no. 20, 2019.
- [2] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [3] Kaggle, “Network traffic anomaly detection dataset,” Kaggle Datasets, 2023, accessed: 2026. [Online]. Available: <https://www.kaggle.com/datasets>
- [4] N. Moustafa and J. Slay, “The UNSW-NB15 dataset for network intrusion detection systems,” in *Military Communications and Information Systems Conference (MilCIS)*. IEEE, 2015.
- [5] Australian Centre for Cyber Security (ACCS), UNSW Canberra, “UNSW-NB15 dataset,” UNSW Research, 2015. [Online]. Available: <https://research.unsw.edu.au/projects/unswnb15-dataset>
- [6] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2019.
- [7] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.
- [8] Canadian Institute for Cybersecurity, University of New Brunswick, “CSE-CIC-IDS2018 dataset,” UNB CIC Datasets, 2018. [Online]. Available: <https://www.unb.ca/cic/datasets/ids-2018.html>
- [9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

- [10] A. Kraskov, H. Stögbauer, and P. Grassberger, “Estimating mutual information,” *Physical Review E*, vol. 69, no. 6, p. 066138, 2004.
- [11] M. A. Ferrag, L. Maglaras, S. Moschogiannis, and H. Janicke, “Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study,” *Journal of Information Security and Applications*, vol. 50, 2020.
- [12] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [13] N. Sameera and M. Shashi, “A review of machine learning-based zero-day attack detection: Challenges and future directions,” *Computer Communications*, vol. 198, pp. 20–35, 2023.
- [14] A. Oliner, A. Ganapathi, and W. Xu, “Advances and challenges in log analysis,” *Communications of the ACM*, vol. 55, no. 2, pp. 55–61, 2012.
- [15] Solarmainframe, “CSE-CIC-IDS2018 (kaggle CSV mirror),” Kaggle Datasets, 2018. [Online]. Available: <https://www.kaggle.com/datasets/solarmainframe/ids-intrusion-csv>
- [16] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Wiley, 2013.
- [17] J. R. Quinlan, “Induction of decision trees,” *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
- [18] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [19] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [20] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.
- [21] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

- [22] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 2008.
- [23] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, “Estimating the support of a high-dimensional distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [24] P. C. Mahalanobis, “On the generalized distance in statistics,” *Proceedings of the National Institute of Sciences of India*, vol. 2, no. 1, pp. 49–55, 1936.
- [25] F. Chollet, *Deep Learning with Python*. Manning Publications, 2018.
- [26] S. J. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. K. Chan, “KDD cup 1999 data,” UCI KDD Archive, Tech. Rep., 1999.
- [27] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009.
- [28] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [29] D. M. W. Powers, “Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation,” *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011.
- [30] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [31] R. Kumar, “Computer log anomaly - detection small dataset code,” https://github.com/RajolKumar2003/Computer-Log-Anomaly-Detection/blob/dataset-code/Small_Dataset.ipynb, 2024, gitHub repository.

- [32] —, “Computer log anomaly detection – medium dataset 1 code,” https://github.com/RajolKumar2003/Computer-Log-Anomaly-Detection/blob/dataset-code/Medium_Dataset_1.ipynb, 2024, gitHub repository.
- [33] I.-K. Yeo and R. A. Johnson, “A new family of power transformations to improve normality or symmetry,” *Biometrika*, vol. 87, no. 4, pp. 954–959, 2000.
- [34] R. Kumar, “Computer log anomaly detection – medium dataset 2 code,” https://github.com/RajolKumar2003/Computer-Log-Anomaly-Detection/blob/dataset-code/Medium_Dataset_2.ipynb, 2024, gitHub repository.
- [35] —, “Computer log anomaly detection – large dataset benign-only code,” https://github.com/RajolKumar2003/Computer-Log-Anomaly-Detection/blob/dataset-code/Large_Dataset_Benign.ipynb, 2024, gitHub repository.
- [36] —, “Computer log anomaly detection – large dataset contaminated training code,” https://github.com/RajolKumar2003/Computer-Log-Anomaly-Detection/blob/dataset-code/Large_Dataset_Contamination.ipynb, 2024, gitHub repository.